

DATA STRUCTURES

LIST
TUPLE
SET
DICTIONARY

List is mutable means add any data type inside
Date: 19-02-2026

.append()

```
a=[10,20,30,40]
b=50
'''
```

```
c="ace".upper()
```

```
a.append(b)
a.append(60)
print(a)
```

```
a.append(c)
print(a)
'''
```

.index()

```
'''
n=[10,20,30]
##m=n.index(20)
print(m)
print(n.index(30))
```

```
a=[10,20,[30,40,[50,60,70,80],92,[12,23,34,[45,56,67,[78,89,90,98],87,76,65],54],43],32,21] # cant
find using .index() cause nested index
```

```
print(a[2][4][3][3][2])
```

```
b=[10,20,[30,[40,50,60,[70,80,90],25,[35,45,55]],65,75],85] # interview question
```

```
print(b[2][1][5][1])
'''
```

.extend()

```
'''
a=[10,20,30,40,50,60]
```

```
#a.extend(80) ## can't cuz of it is int if int inside of list only valid (not only int any data
type insideof list valid)
```

```
a.extend([70]) ## like this
```

```
a.extend(['a','c','e']) ## like this also
```

```
a.extend("luffy") ## it will also add but separate every sinle letters with different index
```

```
a.extend(["sanji"])
```

```

print(a)
"""
### .insert ()
"""
a=[10,20,30]
a.insert(1,15) # insert in the index what we give
a.insert(4,[40,50,60])

print(a)
"""

### replace
"""
b=[10,20,30,40]
b[2] = 50
print(b)
"""

### .sort()

s=[10,60,30,70,50,30]
"""
s.sort()
print(s)

print(s.sort())
"""

### .count()
"""
c=s.count(30)    # how many elements in the list
print(c)
print(s.count(30))
"""

## .remove() based on values
"""
s.remove(50)

print(s)
"""

### .pop()    # based on index
"""
s.pop()        # It will pop last index value
print(s)
s.pop(1)
print(s)
"""

### delete    # del x[:]
"""
d=[1,2,3,4,5]
del d[0:2]      # based on index also but we can give start and end
print(d)
"""

```

```
## .reverse()
"""
r=[10,20,30,40,50]
r.reverse()
print(r)
"""

## .copy()
"""
c=[10,20,30,40]
d=c.copy()
print(d)

c.append([40])
print(c)
print(d)
"""

## .clear()
"""
c=[20,40,60]
#### c.clear(40)          # error -- cause it takes no argument
c.clear()                # clear all values
print(c)
"""
```

##	List	Tuple
#	mutable	immutable
#	dynamic memory	fixed memory
#	many methods	few methods
#	slow	fast because immutable
#	inefficient memory	efficient memory

```
#    ### Tuple
```

```
# IMMUTABLE
# CAN STORE ANY DATA TYPE
# FIXED AND LESS MEMORY
# FASTER THAN LIST
# ALLOW DUPLICATE VALUES
# ORDERED
# ()          using symbol
### store any data type
"""
```

```
a = (10,20,30,40)
print(a)
print(type(a))
```

```

a=(10,"apple",4.5,"bananaa")
print(a)
"""
# tuple unpacking
"""
t=((1,2),(3,4,5))
a,b,c=(t,t,t)
print(a,b,c)
"""
# Allow Duplicate value
"""
a=(10,20,10)
print(a)
"""
# index
"""
a=(10,20,30,40,50)
print(a[0])
print(a[1])
print(a[5])
"""
# index    # nested Tuple
"""
a=([10,20],50)
print(a[0][1])
"""
# .count()
"""
b=a.count(10)
print(b)
"""
# concat
"""
a=(1,2,3)
b=(4,5,6)
print(a+b)
print(b+a)
print(a)
print(b)
"""
#
"""
# Tuple TC to List .append() method and back to TC Tuple
"""
a=(1,2,3,4,5,6)
b=5
c=list(a)
c.append(b)
c.append(8)
print(a)
print(c)
print(tuple(c))
"""

```

```
## type :
```

```
"""  
i=(18)          # int not tuple  
s="Ace"         # string not tuple  
t=(18,"Ace")    # tuple  
print(i, s, t)  
print(type(i), type(s), type(t))  
"""
```

```
## tuple inside tuple
```

```
"""  
t=(1,2,3,(5,6,7),8,9)  
print(t)  
print(type(t))  
print(t[3][2])  
"""
```

```
### SET
```

```
# mutable  
# do not allow duplicate  
# only store {immutable} data type only  
# unordred  
# cant access index  
# {} -- using symbol as like as maths Set
```

```
"""  
a={10,20,30,40}      # here can't  
small={1,3,2,4,6,5}  # if range is low it can order it (sort)  
print(a)             # it will print randomly order  
print(small)  
"""
```

```
#print(a[0]) ---- error
```

```
a={"apple","orange", "mango", "banana"}  
#print(a[0]) -- there is no indexing
```

```
# don't allow duplicate values
```

```
"""  
a={1,2,1,2}  
print(a)
```

```
# list inside set  
b={1,2,3,[4,1,2],6}  # unhashable type: 'list' -- error  
#print(b)  
"""
```

```
# tuple inside set
```

```
"""  
c={1,2,3,(1,2),6}    ### TUPLE is allowed inside of set , cuz TUPLE is IMMUTABLE  
print(c)
```

```

'''
# SET INSIDE SET
'''
##d = {1,2,3,{4,5,6},7}      ### (unhashable type: 'set')
##print(d)
'''

# .update()
'''
a={1,2,3}
b={4,5,10}

b.update(a)      ##### this will add a in b and change b a+b -- with order

print(a)
print(b)
'''

### .remove()
'''
r={1,2,3,4,5}
r.remove(4)      # it works cuz it is based on values
print(r)         # but del and pop() are based on index so these are not works
'''
##r.remove(6)      # error -- cuz it is not there
##print(r)

### .discard()
'''
d={1,2,3,4,5}
d.discard(4)
print(d)

d.discard(6)      # doesn't throw error this is the difference between remove
print(d)
'''
## .clear()
'''
a={1,2,3,4}
a.clear()
print(a)          # clear all values in set so print ---- set{}
'''

### .pop()
'''
p={10,20,30,40,50,60}
p.pop()           # this only works cuz is not indicating index
#p.pop(00)        # It will throw -- error
print(p)
'''
#

```

Venn diagram -- Sets

```
a={1,2,3,4}
b={3,4,5,6}
```

```
# .union()      # | -- union symbol
```

```
"""
```

```
c=a.union(b)    # 1. st method
```

```
print(c)
```

```
print(a|b)      # 2. nd method
```

```
"""
```

```
# .intersection() # & == Ampersand
```

```
"""
```

```
d = a.intersection(b) # 1. st method
```

```
print(d)
```

```
print(a&b) # 2. nd method
```

```
"""
```

```
# .difference()  # - -- minus symbol
```

```
"""
```

```
e = a.difference(b) # 1. st method
```

```
print(e)
```

```
print(a-b)      # 2. nd method
```

```
print(b-a)
```

```
"""
```

```
# .symmetric_difference()
```

```
"""
```

```
f=a.symmetric_difference(b)
```

```
g=b.symmetric_difference(a)
```

```
print(f)
```

```
print(a^b)
```

```
print(b^a)
```

```
"""
```

```
#
```

```
# ASSESSMENT --
```

```
Date: 21-02-2026
```

```
#
```

```
# SUNDAY -- HOLIDAY
```

```
Date: 22-02-2026
```

```
#
```

```
#
```

```
Date: 23-02-2026
```

```
### DICTIONARY -- {}
```

```
# Mutable
```

```
# Key value pair
```

```
# Value can allow duplicate
```

```
# But key can't allow duplicate
```

```
# Value can allow mutable datatype
```

```
# key allow immutable datatype only {NOT MUTABLE DATA}
```

```
# ordered
```

```
#
```

```
a={1:'ace',a:'sabo',joyboy:'luffy',"Monkey D":"Dragon"}
```

```
print(a,type(a))
```

```
# duplicate testing in both key:value
```

```
dv={1:'ace',2:'sabo',3:'ace'}    # Here no problem
```

```
dk={1:'ace',2:'sabo',1:'luffy'}  # {1: 'luffy', 2: 'sabo'} -- reads last dup key only
```

```
print(dv , dk)
```

```
# ordered      ## how we give == op --- not like set
```

```
o={3:'ace',1:'sabo',2:'ace'}
```

```
print(o)
```

```
# Mutable data testing in both key:value
```

```
mv={1:100,a:'sabo',ace:[10,20],4:(5,10),5:{15,30},6:{1:'zoro'}}
```

```
im={(1,2):99}  # -- cuz this is immutable
```

```
#mk={1,2}:'sanji',{3,4}:'tony',{1:'zoro'}:'tony'}  #-- TypeError: cannot use 'list' as a dict key  
(unhashable type: 'list')
```

```
print(mv , im)
```

```
# key index
```

```
a={3:'ace',1:'sabo',2:'luffy'}
```

```
print(a)
```

```
print(a[3])
```

```
#.get()
```

```
b={'a':'zoro',b:'sanji',c:'luffy'}
```

```
#print(b.get('c'),b.get('d'),b.get('e',7))
```

```
# add
```



```

"""
a={3:'ace',1:'sabo',2:'luffy'}
a[4]='zoro'
print(a)
"""

# .()update
"""
a={1:'ace',2:'sabo',3:'luffy'}
b={'a':'zoro','b':'sanji','c':'luffy'}
a.update(b)
print(a)
b.update(a)
print(b)
"""

# x.keys()
"""
k={1:'ace',2:'sabo',3:'luffy'}
b={'a':'zor0','b':'sanji','c':'luffy'}
print(k.keys() , b.keys())
"""

# .values()
"""
v={1:'ace',2:'sabo',3:'luffy'}
print(v.values())
"""

# .items()
"""
a={1:'ace',2:'sabo',3:'luffy'}
print(a.items())
"""

# .clear()
"""
a={1:'ace',2:'sabo',3:'luffy'}
print(a.clear())
"""

# .pop() and .popitem()
"""
a={1:'ace',2:'sabo',3:'luffy'}

#a.pop()                                -- error  cuz need 1 arguement

a.popitem()
print(a)
"""

# delete -- del x[""]
"""
b={'a':'zoro','b':'sanji','c':'luffy'}
del b['b']
print(b)
"""

# .setdefault()    -- c and d is affecting b

b={'a':'zoro','b':'sanji','c':'luffy'}

```

```
c=b.setdefault('d')          # OP: -- None
d=b.setdefault('e','nami')    # OP: -- nami
#print(b,c,d)
```

```
# _____
```

```
#### With-out built-in functions
```

```
# For loop in List          ~
```

```
a=[1,2,3]
b=[4,5,6,7]
```

```
#print(a+b)                ~
```

```
"""
for i in range(len(b)):
a.append(b[i])
print(a)
"""
```

```
# remove values in List without method
```

```
r=[10,20,30,40]
s=40
c=list([])
#r.remove(50)
#print(r)
"""
```

```
for i in range(len(r)):
if s == r[i]:
continue
else:
c.append(r[i])
print(c)
"""
```

```
# rev
```

```
"""
for i in range(len(r)-1,-1,-1):
c.append(r[i])
print(c)
"""
```

```
# _____
```

```
# _____
```

